R workshop part 5 - Projects and data import

Jan Gačnik

Prepared: 2026-10-26

Paths

Paths are directions that tell R where to look for data or place outputs on our computer.

- Example Windows path: `C:\Users\gacnik\Desktop`
- Example Mac path: `/Users/gacnik/Desktop`

R only accepts paths with forward slash character (`/`), meaning that:

- Windows paths cannot be used as is, replacement with `/` characters
- Mac paths can be used as is

Paths

Use `getwd()` to **get** your *current* R working directory. This is where R currently looks for data and saves outputs to.

```
1 getwd()
```

```
[1] "C:/Users/gacnik/OneDrive - ijs.si/Jan 2023+/R course/r-data-workshop"
```

Use `setwd()` to **set** your *new* R working directory.

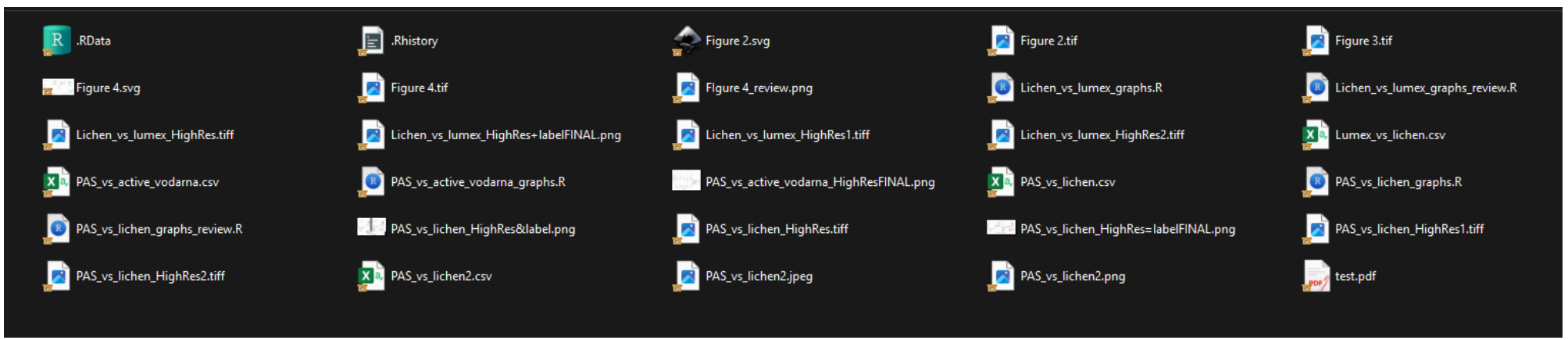
```
1 setwd("C:/ENTER/YOUR/PATH/HERE")
```

There is an easier and better way to manage paths through **RStudio projects**

Organizing your analysis project

As your data analysis skill improve, tasks become larger, and supporting files start growing.

- We started our work in an R script, then we generated some plots, tables… if we started saving these outputs and adding new data, things could get disorganized fast:



Personal archive, my first messy R project

Organizing your analysis project

Project organization for larger tasks/projects:

- everything in **one project folder**, with standard **sub folders**
  - subfolders `data`, `scripts`, `outputs`, ...
- use of **RStudio project file** (`.Rproj` file)
  - relative instead of absolute paths
  - preserves specific rstudio environment and setup just for the project

(Showcase the use of RStudio project)

Relative paths make projects portable

We said RStudio project uses relative paths, not absolute, what does it mean?

- **Absolute path:** starts from a fixed location on one computer
  `C:/Users/gacnik/Desktop/example_project/data/penguins.csv`
- **Relative path:** starts from the project folder
  `data/penguins.csv`

Rule: Always use relative paths in project scripts. Avoid hard-coded absolute paths: they usually break when a project is moved, shared, or opened on another computer.

Data import

Working with data already provided by R was neat, but you will need to apply the learned concept on your data.

Here, we will focus on plain-text rectangular files such as CSV, TSV, and Excel files

```
Code,Station name,Units,Observed conc,Modelled conc
DE0002R,Waldhof,ng/m3,1.58,1.23
DE0003R,Schauinsland,ng/m3,1.24,1.21
DE0009R,Zingst,ng/m3,1.45,1.29
EE0009R,Lahemaa,ng/m3,1.10,1.25
FI0027R,Virolahti Harju,ng/m3,1.16,1.22
FI0050R,Hyytiala,ng/m3,1.24,1.18
GB0048R,Auchencorth Moss,ng/m3,1.29,1.22
GB1055R,Chilbolton Observatory,ng/m3,1.42,1.28
NO0042G,Zeppelin mountain ,ng/m3,1.53,1.13
NO0073U,Sofienbergparken,ng/m3,1.50,1.18
PL0005R,Diabla Gora,ng/m3,1.37,1.19
SE0014R,Rao,ng/m3,1.26,1.26
```

Example of a simple CSV file

Data import

Working with data already provided by R was neat, but you will need to apply the learned concept on your data.

Here, we will focus on plain-text rectangular files such as CSV, TSV, and Excel files

Code	Station name	Units	Observed conc	Modelled conc
DE0002R	Waldhof	ng/m3	1.58	1.23
DE0003R	Schauinsland	ng/m3	1.24	1.21
DE0009R	Zingst	ng/m3	1.45	1.29
EE0009R	Lahemaa	ng/m3	1.1	1.25
FI0027R	Virolahti Harju	ng/m3	1.16	1.22
FI0050R	Hyytiala	ng/m3	1.24	1.18
GB0048R	Auchencorth Moss	ng/m3	1.29	1.22
GB1055R	Chilbolton Observatory	ng/m3	1.42	1.28
NO0042G	Zeppelin mountain	ng/m3	1.53	1.13
NO0073U	Sofienbergparken	ng/m3	1.5	1.18
PL0005R	Diabla Gora	ng/m3	1.37	1.19
SE0014R	Rao	ng/m3	1.26	1.26

Same example, viewed as a table

Data import with a relative path

1. Download and extract the prepared **example_project RStudio project**.

2. Open `example_project.Rproj`. Then import the file with `read_csv()` using a relative path:

```
1 library(tidyverse)
2
3 mercury_air <- read_csv("data/mercury_air.csv")
```

```
# A tibble: 12 × 5
  Code    `Station name`   Units `Observed conc` `Modelled conc`
  <chr>   <chr>            <chr> <chr>                    <dbl>
1 DE0002R Waldhof          ng/m3 1.58                      1.23
2 DE0003R N/A              ng/m3 1.24                      1.21
3 DE0009R Zingst           ng/m3 1.45                      1.29
4 EE0009R Lahemaa          ng/m3 N/A                       1.25
5 FI0027R Virolahti Harju  ng/m3 1.16                      1.22
6 FI0050R N/A              ng/m3 1.24                      1.18
7 GB0048R Auchencorth Moss ng/m3 1.29                      1.22
# ℹ 5 more rows
```

Data import

`read_csv()` function read the N/A cells as a text, not as a true `NA` value. We need to define what represents `NA` values using the `na` argument:

```
mercury_air <- read_csv("data/mercury_air.csv", na = c("N/A", ""))
mercury_air
```

```
# A tibble: 12 × 5
   Code    `Station name`          Units `Observed conc` `Modelled conc`
   <chr>   <chr>                   <chr>           <dbl>           <dbl>
 1 DE0002R Waldhof                 ng/m3            1.58            1.23
 2 DE0003R <NA>                    ng/m3            1.24            1.21
 3 DE0009R Zingst                  ng/m3            1.45            1.29
 4 EE0009R Lahemaa                 ng/m3           NA               1.25
 5 FI0027R Virolahti Harju         ng/m3            1.16            1.22
 6 FI0050R <NA>                    ng/m3            1.24            1.18
 7 GB0048R Auchencorth Moss        ng/m3            1.29            1.22
 8 GB1055R Chilbolton Observatory  ng/m3            1.42            1.28
 9 NO0042G Zeppelin mountain       ng/m3            1.53            1.13
10 NO0073U Sofienbergparken        ng/m3            1.5             1.18
11 PL0005R Diabla Gora             ng/m3            1.37            1.19
12 SE0014R Rao                     ng/m3            1.26            1.26
```

Data import

The names of columns have white space, and as it is annoying to manually rename each one using the `rename()` function. We can do it using the janitor package function `clean_names()`:

```
library(janitor)

mercury_air <- read_csv("data/mercury_air.csv", na = c("N/A", "")) %>%
  clean_names()

mercury_air
```

```
# A tibble: 12 × 5
   code    station_name           units observed_conc modelled_conc
   <chr>   <chr>                  <chr>         <dbl>         <dbl>
 1 DE0002R Waldhof                ng/m3          1.58          1.23
 2 DE0003R <NA>                   ng/m3          1.24          1.21
 3 DE0009R Zingst                 ng/m3          1.45          1.29
 4 EE0009R Lahemaa                ng/m3         NA             1.25
 5 FI0027R Virolahti Harju        ng/m3          1.16          1.22
 6 FI0050R <NA>                   ng/m3          1.24          1.18
 7 GB0048R Auchencorth Moss       ng/m3          1.29          1.22
 8 GB1055R Chilbolton Observatory ng/m3          1.42          1.28
 9 NO0042G Zeppelin mountain      ng/m3          1.53          1.13
10 NO0073U Sofienbergparken       ng/m3          1.5           1.18
11 PL0005R Diabla Gora            ng/m3          1.37          1.19
12 SE0014R Rao                    ng/m3          1.26          1.26
```

Data import

Depending on the mess level of your data files, more arguments of `read_csv()` could be needed, for example:

- using the `skip` argument to tell `read_csv()` how many metadata headers in files to **skip**
- using the `col_types` argument to explicitly tell `read_csv()` what type each column should be, instead of relying on guessing
- using one of `parse_number()`, `parse_date()`, `parse_time()`, … functions to **convert values in columns to their correct type**

Import several similar files at once

The `data/` folder also contains similarly structured files per year: `mercury_air_2022.csv`, `mercury_air_2023.csv`, `mercury_air_2024.csv`

- Let's make a vector - `c()` - of file names, and pass it to the `read_csv()`:

```
mercury_files <- c("data/mercury_air_2022.csv",
           "data/mercury_air_2023.csv",
           "data/mercury_air_2024.csv")

mercury_air_all <- read_csv(mercury_files, id = "file_name")
mercury_air_all
```

Import several similar files at once

The result is a data frame which contains data from all three CSV files:

```
# A tibble: 36 × 7
   file_name     Year Code  `Station name` Units `Observed conc` `Modelled conc`
   <chr>        <dbl> <chr> <chr>          <chr> <chr>                      <dbl>
 1 data/mercur…  2022 DE00… Waldhof        ng/m3 1.55                        1.2
 2 data/mercur…  2022 DE00… N/A            ng/m3 1.18                        1.17
 3 data/mercur…  2022 DE00… Zingst         ng/m3 1.4                         1.26
 4 data/mercur…  2022 EE00… Lahemaa        ng/m3 1.08                        1.21
 5 data/mercur…  2022 FI00… Virolahti Har… ng/m3 1.12                        1.18
 6 data/mercur…  2022 FI00… N/A            ng/m3 1.2                         1.15
 7 data/mercur…  2022 GB00… Auchencorth M… ng/m3 1.25                        1.18
 8 data/mercur…  2022 GB10… Chilbolton Ob… ng/m3 1.38                        1.24
 9 data/mercur…  2022 NO00… Zeppelin moun… ng/m3 1.49                        1.1
10 data/mercur…  2022 NO00… Sofienbergpar… ng/m3 1.46                        1.16
11 data/mercur…  2022 PL00… Diabla Gora    ng/m3 N/A                         1.16
12 data/mercur…  2022 SE00… Rao            ng/m3 1.22                        1.23
13 data/mercur…  2023 DE00… Waldhof        ng/m3 1.58                        1.23
14 data/mercur…  2023 DE00… N/A            ng/m3 1.24                        1.21
15 data/mercur…  2023 DE00… Zingst         ng/m3 1.45                        1.29
16 data/mercur…  2023 EE00… Lahemaa        ng/m3 N/A                         1.25
17 data/mercur…  2023 FI00… Virolahti Har… ng/m3 1.16                        1.22
```

Data export

Data export or *data writing* is the reverse process of data import. Data can be saved to CSV or TSV using `write_csv()` and `write_tsv()`, respectively.

- Two arguments are used: `x` - data frame to be saved, and `file` - the location where to save it:

```
1 write.csv(mercury_air_all, "outputs/mercury_air_all.csv")
```

Note that the file was saved into the `outputs` sub directory of our project directory

- Handy for saving text file outputs, figures, models, or any other output results.

Data entry

Sometimes we need to make a table “by hand” using `data.frame()`. This can be done by data entry in your R script, although it is not recommended for larger data.

- The layout follows this structure: column_name1 = c(value1, value2, value3), column_name2 = c(value4, value5, value6), … = …

```r
data.frame(
  x = c(1, 2, 4),
  y = c("dog", "cat", "pig"),
  z = c(0.02, 0.05, 0.08)
)
```

```
  x   y    z
1 1 dog 0.02
2 2 cat 0.05
3 4 pig 0.08
```

Part 1 · Introduction to R (1/2)

Work in a script

- Run the current line with `Ctrl` + `Enter`
- Use `#` comments to explain **why**
- Use `?function_name` when you need help
- Read errors from the first line and check brackets, commas, and spelling

Think in functions and objects

- Functions perform actions: `mean(x)`
- Arguments control them: `mean(x, na.rm = TRUE)`
- `<-` stores a result in an object
- Clear names make later steps easier to follow

Part 1 · Introduction to R (2/2)

```r
# Install once; load each session
install.packages("tidyverse")
library(tidyverse)

mass <- c(4200, 3900, 4600)
mean_mass <- mean(mass)
```

- A vector contains values of one basic type
- A data frame organizes vectors into columns
- Use `object[row, column]` to subset a data frame
- Installing adds a package; loading makes its functions available
- Reassigning an existing name overwrites the object

Part 2 · Data visualization (1/2)

Data → aesthetic mappings → geometric layers

```
ggplot(
  data = penguins,
  mapping = aes(
    x = body_mass,
    y = flipper_len,
    color = species
  )
) +
  geom_point()
```

- Put variables inside `aes()`
- Put fixed choices outside `aes()`
- Add layers with `+`
- Choose a geometry that answers the question
- Add labels and a theme only after the plot communicates clearly

Part 2 · Data visualization (2/2)

What do you want to see?

- One numeric variable → histogram or density plot
- Numeric values across groups → boxplot
- Two numeric variables → scatter plot
- Patterns within groups → color, shape, or facets

```
penguins %>%
  ggplot(aes(
    x = body_mass,
    y = flipper_len,
    color = species
  )) +
  geom_point() +
  facet_wrap(~ sex) +
  labs(
    x = "Body mass (g)",
    y = "Flipper length (mm)"
  ) +
  theme_minimal()
```

Save the final plot with `ggsave()`; set its filename, size, and resolution.

Part 3 · Data transformation (1/2)

Goal	Main verbs
Keep rows	`filter()`, `slice()`, `distinct()`
Work with columns	`select()`, `rename()`, `mutate()`
Count observations	`count()`
Calculate by group	`group_by()` + `summarise()`

Before summarizing: check missing values; use `na.rm = TRUE` when removal is appropriate.

Part 3 · Data transformation (2/2)

```r
penguins %>%
  filter(!is.na(sex), !is.na(body_mass)) %>%
  mutate(body_mass_kg = body_mass / 1000) %>%
  group_by(species, sex) %>%
  summarise(
    mean_mass_kg = mean(body_mass_kg),
    n = n(),
    .groups = "drop"
  )
```

Read top to bottom: **start with data**, then add one clear step per line.

Part 4 · Data tidying (1/2)

- Each **variable** is a column
- Each **observation** is a row
- Each **value** is one cell

Tidy data makes it easier to:

- transform with `dplyr`
- visualize with `ggplot2`
- compare observations
- reuse code across variables and groups

Part 4 · Data tidying (2/2)

Wide → long

```
billboard %>%
  pivot_longer(
    cols = starts_with("wk"),
    names_to = "week",
    values_to = "rank",
    values_drop_na = TRUE
  )
```

Long → wide

```
Orange %>%
  pivot_wider(
    names_from = age,
    values_from = circumference,
    names_prefix = "age_days_"
  )
```

Use consistent spacing and indentation; break long pipelines and named arguments across lines.

Part 5 · Projects and data import

```
my_project/
├─ my_project.Rproj
├─ data/
├─ scripts/
└─ outputs/
```

- Open the `.Rproj` file before working
- Keep inputs, scripts, and outputs in predictable folders
- Use relative paths such as `data/file.csv`
- Avoid paths tied to one person or computer
- Treat original data as read-only

Test: move the project folder; the scripts should still run.

Final project

We have covered the core R concepts. Now, let’s use a real environmental dataset to demonstrate a complete data-analysis workflow in R.

Part 6 - Complete project demonstration